KOTLIN INTERVIEW PREPARATION

Connect Kotlin terminology to your Java + Spring Boot experience

Your starting point
Java 17 · Spring Boot · Hibernate/JPA · REST APIs · JUnit · Spring Security

Your goal
Explain the Kotlin code in this Books & Authors GraphQL API clearly, accurately, and in terms you already understand.

How to use this guide

Read the Java-to-Kotlin comparison first. Then rehearse the short project explanation and interview answers. You do not need to claim experience with Kotlin features you have not used: explain the code in front of you, connect it to Java/Spring concepts you know, and be candid about what is new.

Project stack: Kotlin 2.2.21 · Java 17 · Spring Boot 3.5.16 · Spring for GraphQL · Spring Data JPA/Hibernate · H2 · Gradle Kotlin DSL.

Contents

- 1. Your experience is a strong bridge
- 2. Kotlin syntax in Java terms
- 3. Types, nullability, and data classes
- 4. Collections, lambdas, and generics
- 5. Spring Boot and JPA with Kotlin
- 6. GraphQL implementation walkthrough
- 7. Interview questions with clear answers
- 8. A short demo and speaking script
- 9. Terms to remember

1. Your experience is a strong bridge

You already know much of the ecosystem around this Kotlin application. The main learning step is recognizing Kotlin syntax and conventions around familiar JVM and Spring ideas.

Your experience	How it connects here
Java 17 / Spring Boot	Same JVM and Spring lifecycle; Kotlin is the application language.
Hibernate / Spring Data JPA	Same entity mapping, repositories, transactions, and persistence context.
REST APIs	GraphQL is a different API contract; Spring still routes requests to resolver methods.
JUnit / integration testing	Tests use JUnit 5 and Spring GraphQL's GraphQlTester.
Spring Security	Resume experience is relevant generally; this demo app has no configured authentication or authorization.

Interview framing: "The framework and persistence ideas are familiar from my Java/Spring work. Kotlin changes the language syntax and gives stronger null-safety and concise data handling; I can show exactly how those appear in this project."

2. Kotlin syntax in Java terms

Kotlin is a statically typed JVM language. It compiles to JVM bytecode and interoperates with Java libraries, including Spring and Hibernate. Think of the examples below as Kotlin spellings for concepts you already use.

Read-only and mutable references: val and var

```
val authorId = "author-7"   // cannot be reassigned
var title = "Original"       // can be reassigned
title = "Revised"
```

Java comparison: `val` is similar to a local `final` reference; `var` is a normal mutable local. Neither means the referenced object itself is immutable.

Type inference and explicit types

```
val page = 0                // inferred Int
val book: Book? = findBook() // explicit nullable type
```

Kotlin often infers obvious local types, while public APIs and domain boundaries benefit from explicit types. Java developers often write the equivalent type on every local declaration.

String templates and named arguments

```
val message = "Book ${book.id}: ${book.title}"
Book(id = "b-1", title = "Kotlin", author = author)
```

String templates replace many Java concatenations or `String.format` calls. Named arguments make constructor calls self-documenting; Java does not have named arguments.

3. Types, nullability, and data classes

Null safety is one of Kotlin's most useful differences. A Kotlin type such as `String` is non-null by default; `String?` explicitly permits null. This is checked by the compiler and makes absence part of the type.

```
fun findById(id: String): Book? =
    bookRepository.findById(id).orElse(null)
```

Java connection: think of `Book?` as an explicit nullable return, similar in intent to `Optional<Book>` or a documented nullable Java return. It is not the same runtime wrapper as Optional: Kotlin's nullable reference is still a normal JVM reference.

In the GraphQL schema, `bookById` returns nullable `Book`, while `Book.author: Author!` is required. The schema and Kotlin types should tell a consistent story about absence.

Data classes

```
data class BookInput(
    val title: String,
    val authorId: String,
)
```

A Kotlin `data class` automatically provides value-style methods such as `equals`, `hashCode`, `toString`, and `copy`. Compare it to a Java record or a POJO with generated methods. Use data classes for input/value objects; this project keeps JPA entities as regular classes because ORM entities have identity, mutability, proxies, and lifecycle concerns.

Properties and accessors

Kotlin properties look like fields at the call site, but typically compile to JVM fields plus getters/setters. For example, `book.title` is property syntax; Java/Spring tooling sees normal JVM accessors where generated.

4. Collections, lambdas, and generics

Kotlin collection operations often express transformations directly. They are related in purpose to Java Streams, but they are Kotlin standard-library APIs and are not Java Stream pipelines.

```
val ids = books.map { it.author.id }

val authorsById = authors
    .associateBy(Author::id)

val result: Map<Book, Author> = books.associateWith { book ->
    authorsById[book.author.id]
        ?: error("Book references a missing Author")
}
```

`map` transforms each element. `associateBy` makes a map indexed by a key. `associateWith` creates a map from each input Book to a computed Author. The `?:` operator is the Elvis operator: use the value on the left, otherwise evaluate the right side.

Interfaces and generics

```
interface BookRepository : JpaRepository<Book, String>
```

This is a Kotlin interface extending a Java Spring Data interface. The generic arguments mean “repository for Book entities whose ID type is String.” Spring Data creates the repository implementation at runtime, just as in a Java project.

Lambda parameter `it`

When a lambda has one parameter and its name is omitted, Kotlin exposes it as `it`. Prefer a named parameter when the transformation is complex or the lambda has nested scopes.

5. Spring Boot and JPA with Kotlin

Dependency injection, annotations, repositories, and transactions are the Spring concepts you know. Kotlin classes and methods are final by default, while some Spring proxy features and ORM proxies expect types that can be subclassed. This project uses the Kotlin Spring plugin and marks JPA entity classes/properties `open`.

```
@Service
class BookQueryService(
    private val bookRepository: BookRepository,
) {
    @Transactional(readOnly = true)
    fun findById(id: String): Book? =
        bookRepository.findById(id).orElse(null)
}
```

Constructor injection

The constructor parameter is the dependency declaration. Spring injects `BookRepository` when it creates the service. Kotlin's primary constructor keeps this concise; conceptually it corresponds to a Java constructor taking the repository and assigning it to a field.

JPA relationship

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "author_id", nullable = false)
open lateinit var author: Author
```

This is the owning Book-to-Author relationship. Many Books may reference one Author; the foreign key is non-null. `lateinit` means the property is assigned after construction, here by persistence/framework code. It is not a general replacement for good initialization; accessing it before assignment throws.

Transactions

`@Transactional` defines a unit of database work through Spring. A read-only transaction documents read intent; a mutation transaction lets changes commit together or roll back on failure, depending on exception and transaction configuration.

6. GraphQL implementation walkthrough

GraphQL changes the API contract, not the basic Spring programming model: annotated resolver methods route schema fields to services. A client chooses which fields it needs. The project uses GraphiQL at `/graphiql` and GraphQL HTTP at `/graphql`.

```
type Query {
  books(page: Int! = 0, size: Int! = 20): [Book!]!
  bookById(id: ID!): Book
  authors: [Author!]!
}

type Book { id: ID!, title: String!, author: Author! }
```

`!` means non-null in SDL. `[Book!]!` means a non-null list containing non-null Books. The pagination signature is the project's provisional contract; the assignment asks for pagination but does not mandate these exact arguments.

Resolver and batch relationship

```
@QueryMapping
fun books(@Argument page: Int, @Argument size: Int): List<Book> =
    bookQueryService.findPage(page, size)

@BatchMapping(typeName = "Book", field = "author")
fun bookAuthors(books: List<Book>): Map<Book, Author> {
    // collect author IDs, load them together, map them back to Books
}
```

`@QueryMapping` resolves the root query. `@Argument` binds GraphQL arguments. `@BatchMapping` resolves a field for a group of parent objects; here it loads Authors together rather than causing a separate Author lookup for each Book.

Mutation input and omitted versus null

Kotlin nullable properties alone cannot always express whether a GraphQL input field was omitted or explicitly sent as null. The update input uses Spring GraphQL `ArgumentValue<T>` so the service can preserve an omitted field but reject explicit null for the required Author relation. This is framework support used from Kotlin, not a Kotlin language keyword.

7. Interview questions with clear answers

1. Is the project written in Java?

No. It is written in Kotlin and runs on the JVM. Java libraries and Spring APIs are used through JVM interoperability.

2. What is the difference between val and var?

val cannot be reassigned; var can. Neither alone guarantees deep immutability of an object.

3. What does String? mean?

It is a nullable String. The question mark makes possible absence explicit in the type.

4. How is Book? related to Optional<Book>?

Both communicate possible absence. Kotlin Book? is a nullable reference, not an Optional wrapper.

5. What does a data class provide?

Generated value-oriented methods such as equality, hash code, string representation, and copy.

6. Would you make a JPA entity a data class?

Usually not by default. ORM entities have identity, mutable persistence state, proxies, and lazy associations that can interact badly with generated equality and copying.

7. What are map and associateBy doing?

map transforms elements; associateBy creates a map keyed by a selected property.

8. Are Kotlin collection functions Java Streams?

No. They are Kotlin standard-library operations with similar use cases; their APIs and evaluation behavior differ.

9. What does JpaRepository<Book, String> say?

It is a Spring Data repository for Book entities with String IDs.

10. Why are the JPA entity classes open?

Kotlin types are final by default. Hibernate proxying and framework subclassing can need open types; this project uses open entities and the Spring plugin.

11. What does lateinit mean?

The property is non-null in its declared type but will be assigned later. Accessing it before initialization throws.

12. Why use constructor injection?

It makes required dependencies explicit, supports immutable references, and works cleanly with Spring component creation.

13. What does @Transactional do?

It lets Spring manage a transaction around the method so related database work commits or rolls back together.

14. How does @BatchMapping help?

It resolves a nested field for multiple parent objects together, allowing one grouped Author lookup instead of one lookup per Book.

15. What is GraphQL nullability?

The SDL ! marker means a value cannot be null. It is part of the API contract and affects how clients handle responses.

16. How do you distinguish omitted input from explicit null?

Use Spring GraphQL ArgumentValue, which represents omission separately from a supplied value that may itself be null.

17. What evidence shows N+1 improvement?

The regression test counts SQL during GraphQL execution for two result sizes, separating root Book selects from Author relationship selects. Nested response data by itself is not query-count evidence.

18. Is the app secured with Spring Security?

No. This learning/demo project has no Spring Security configuration. I would not present it as production-secured.

19. What would you improve next for production?

Choose an authentication and authorization design, durable database and migrations, operational configuration, and security review based on requirements.

20. What if you don’t know a Kotlin feature?

I would say I have not used it yet, explain what I can infer from the code and Java equivalent, then verify the details rather than guess.

8. A short demo and speaking script

Opening (30 seconds)

"I work primarily with Java and Spring, and this project applies familiar Spring and JPA patterns in Kotlin. I'll show the API contract, a Book–Author workflow, then explain the Kotlin and batching choices."

Walkthrough (3–5 minutes)

- Open GraphiQL and run authors to show the demo-profile Author.
- Add a Book with that existing Author ID; use the returned Book ID.
- Query books with id, title, and nested author { id name }.
- Update the title while omitting authorId; point out that the relationship is preserved.
- Delete the Book, then query authors again to show the Author remains.
- Explain @BatchMapping and point to the SQL-count test separately from the response test.

Close honestly (30 seconds)

"This is a learning/demo API using volatile H2 storage. The pagination shape is a project decision, and authentication is not configured. The suite has previously recorded passing results; I will distinguish those records from any test run performed today."

Good interview habit: State the behavior, point to the code, then explain the trade-off. If a choice is provisional or outside this project, say so plainly.

9. Terms to remember

JVM — Java Virtual Machine; runs bytecode produced by Java and Kotlin compilers.

Nullability — Whether a value may be absent; Kotlin expresses this in the type with ?.

Data class — Kotlin class with generated value-style methods.

Property — Kotlin field-like syntax backed by JVM accessors as needed.

Lambda — A function value passed to another function.

Generic — A type parameter, such as Book and String in JpaRepository<Book, String>.

SAM conversion — Kotlin convenience for passing a lambda to a Java single-abstract-method interface.

Proxy — A framework-created wrapper/subclass used for behavior such as transactions or lazy ORM loading.

Persistence context — JPA-managed set of entity instances and their tracked changes.

Lazy loading — Loading an association when it is accessed rather than with the initial entity query.

Resolver — Method that supplies a GraphQL field value.

Batch mapping — Resolving a nested GraphQL field for several parent objects together.

N+1 — A query pattern where one root query is followed by one relationship query per returned row.

ArgumentValue — Spring GraphQL type that distinguishes omitted input from explicitly supplied null/value.

Provisional contract — A project-selected API choice not mandated by the assignment material.

Grounding note: examples and project claims in this guide are based on the current Books & Authors repository. The resume describes your broader experience; use your own examples when discussing work outside this project.